

Implement a Deep Convolutional GAN to generate complex color images

27/10/2025 Lab 12 :

Aim : To implement and train a Deep Convolutional GAN (Generative Adversarial Network) to generate complex color images

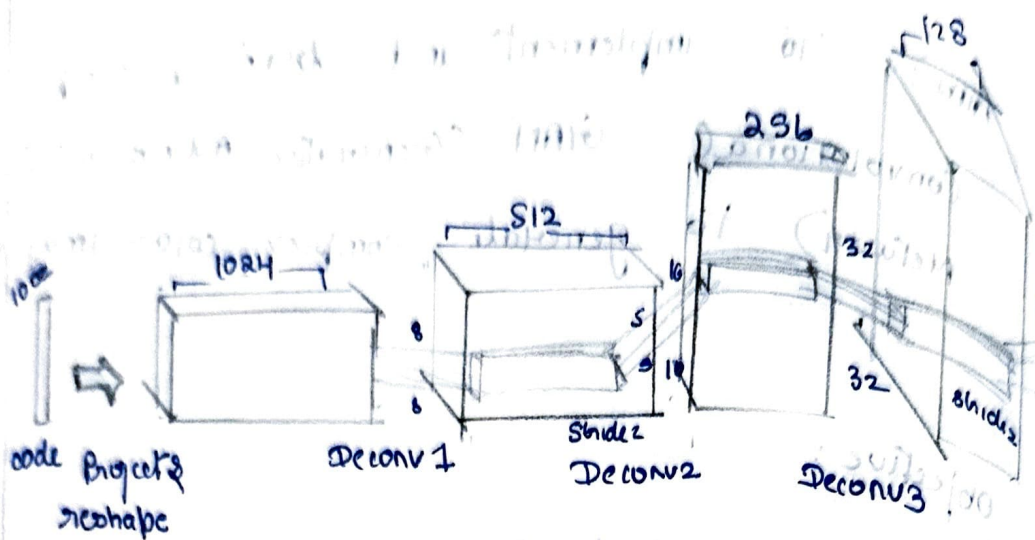
Objective :

- ★ To understand GAN architecture
- ★ To train model on color image dataset
- ★ To observe training dynamics and quality of generating images.

Pseudocode :

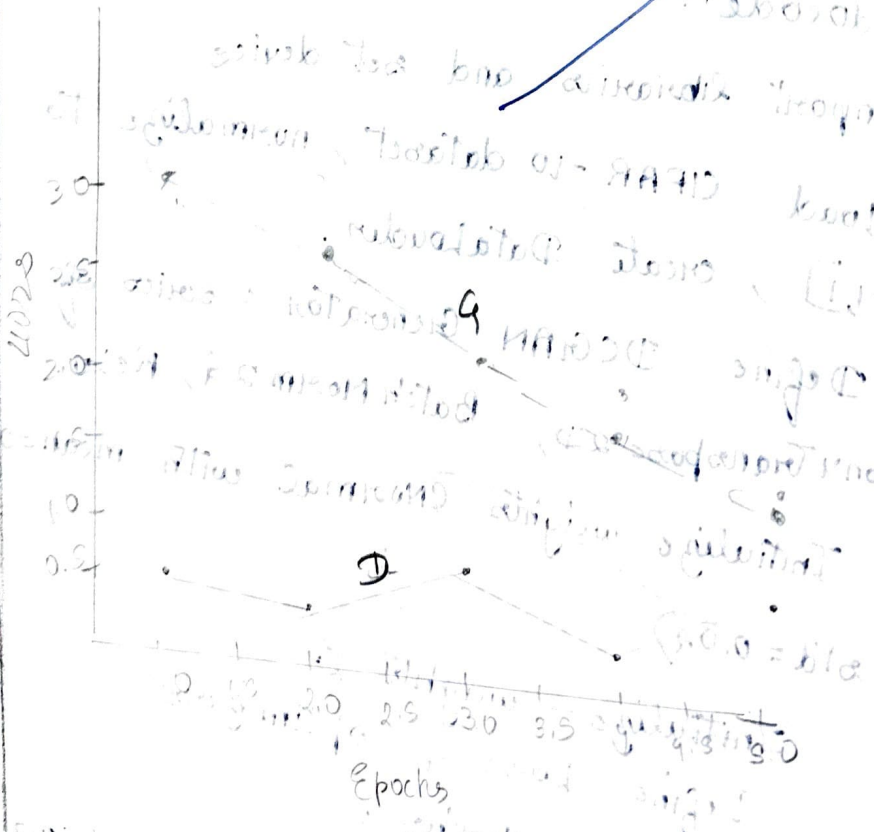
- 1) Import libraries and set device
- 2) Load CIFAR-10 dataset, normalize to $[-1, 1]$, create DataLoader.
- 3) Define DCGAN Generator : series of convTranspose 2D, Batch Norm 2d, ReLU.
- 4) Initialize weights (Normal with mean=0, std=0.02)
- 5) ~~Initialize weights &~~
Define Loss & optimizers
- 6) For each epochs :
 - a) Train Discriminator on real images
 - b) Train Generator to fool Discriminator
 - c) Save generator a batch of generated image for visualization.
- 7) Monitor losses

DCGAN architecture



Q.P

Epoch 1/5	Loss D: 0.8924	Loss G: 2.9612
Epoch 2/5	Loss D: 0.5894	Loss G: 2.6662
Epoch 3/5	Loss D: 0.7406	Loss G: 2.3580
Epoch 4/5	Loss D: 0.5590	Loss G: 2.0222
Epoch 5/5	Loss D: 1.3862	Loss G: 1.5855



Observations

- * The discriminator and generator losses usually fluctuate this is expected in GAN training : as one improves, other responds
- * Initially generator images are noisy random patterns.
- * over epochs starts showing structure (colors, shape)
- * Saving images each ~~shape~~ epoch helps visualize progression from noise $\rightarrow$ structure $\rightarrow$ clearer images.

Result

Successfully

Implemented DCGAN

[1]
✓ 13s

```
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
import torchvision.utils as vutils
import matplotlib.pyplot as plt
import numpy as np
```

[2]
✓ 0s

```
batch_size = 128
image_size = 32
nz = 100
ngf = 64
ndf = 64
nc = 3
lr = 0.0002
beta1 = 0.5
epochs = 5
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

[3]
✓ 0s

```
transform = transforms.Compose([
    transforms.Resize(image_size),
    transforms.ToTensor(),
```




```
fake = netG(noise)
```

```
output_fake = netD(fake.detach())
```

```
lossD_fake = criterion(output_fake, label_fake)
```

```
lossD = lossD_real + lossD_fake
```

```
lossD.backward()
```

```
optimizerD.step()
```

```
# --- Train Generator ---
```

```
netG.zero_grad()
```

```
output = netD(fake)
```

```
lossG = criterion(output, label_real)
```

```
lossG.backward()
```

```
optimizerG.step()
```

```
print(f"Epoch [{epoch+1}/{epochs}]  Loss_D: {lossD.item():.4f}  Loss_G: {lossG.item():.4f}")
```

```
# Show sample results
```

```
with torch.no_grad():
```

```
    fake = netG(fixed_noise).detach().cpu()
```

```
grid = vutils.make_grid(fake, padding=2, normalize=True)
```

```
plt.figure(figsize=(8,8))
```

```
plt.axis("off")
```

```
plt.title(f"Generated Images at Epoch {epoch+1}")
```

```
plt.imshow(np.transpose(grid, (1,2,0)))
```

```
plt.show()
```

[3]

✓ 0s



```
transform = transforms.Compose([
    transforms.Resize(image_size),
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
])
```

[4]

✓ 9s

```
dataset = datasets.CIFAR10(root="./data", download=True, transform=transform)
dataloader = DataLoader(dataset, batch_size=batch_size, shuffle=True)
```

100%|██████████| 170M/170M [00:05<00:00, 28.9MB/s]

[5]

✓ 0s

```
class Generator(nn.Module):
    def __init__(self):
        super(Generator, self).__init__()
        self.main = nn.Sequential(
            nn.ConvTranspose2d(nz, ngf*8, 4, 1, 0, bias=False),
            nn.BatchNorm2d(ngf*8),
            nn.ReLU(True),
            nn.ConvTranspose2d(ngf*8, ngf*4, 4, 2, 1, bias=False),
            nn.BatchNorm2d(ngf*4),
            nn.ReLU(True),
            nn.ConvTranspose2d(ngf*4, ngf*2, 4, 2, 1, bias=False),
            nn.BatchNorm2d(ngf*2),
            nn.ReLU(True)
```

[5]
✓ Os



```
class Generator(nn.Module):
    def __init__(self):
        super(Generator, self).__init__()
        self.main = nn.Sequential(
            nn.ConvTranspose2d(nz, ngf*8, 4, 1, 0, bias=False),
            nn.BatchNorm2d(ngf*8),
            nn.ReLU(True),
            nn.ConvTranspose2d(ngf*8, ngf*4, 4, 2, 1, bias=False),
            nn.BatchNorm2d(ngf*4),
            nn.ReLU(True),
            nn.ConvTranspose2d(ngf*4, ngf*2, 4, 2, 1, bias=False),
            nn.BatchNorm2d(ngf*2),
            nn.ReLU(True),
            nn.ConvTranspose2d(ngf*2, ngf, 4, 2, 1, bias=False),
            nn.BatchNorm2d(ngf),
            nn.ReLU(True),
            nn.ConvTranspose2d(ngf, nc, 3, 1, 1, bias=False),
            nn.Tanh()
        )

    def forward(self, input):
        return self.main(input)
```

[6]
4.8

```
class Discriminator(nn.Module):
```

[6]

✓ Os



```
class Discriminator(nn.Module):
    def __init__(self):
        super(Discriminator, self).__init__()
        self.main = nn.Sequential(
            nn.Conv2d(nc, ndf, 4, 2, 1, bias=False),
            nn.LeakyReLU(0.2, inplace=True),
            nn.Conv2d(ndf, ndf*2, 4, 2, 1, bias=False),
            nn.BatchNorm2d(ndf*2),
            nn.LeakyReLU(0.2, inplace=True),
            nn.Conv2d(ndf*2, ndf*4, 4, 2, 1, bias=False),
            nn.BatchNorm2d(ndf*4),
            nn.LeakyReLU(0.2, inplace=True),
            nn.Conv2d(ndf*4, 1, 4, 1, 0, bias=False),
            nn.Sigmoid()
        )

    def forward(self, input):
        return self.main(input).view(-1)
```

[7]

✓ Os

```
netG = Generator().to(device)
netD = Discriminator().to(device)

criterion = nn.BCELoss()
optimizerD = optim.Adam(netD.parameters(), lr=lr, betas=(beta1, 0.999))
```



```
netG = Generator().to(device)
netD = Discriminator().to(device)

criterion = nn.BCELoss()
optimizerD = optim.Adam(netD.parameters(), lr=lr, betas=(beta1, 0.999))
optimizerG = optim.Adam(netG.parameters(), lr=lr, betas=(beta1, 0.999))

fixed_noise = torch.randn(64, nz, 1, 1, device=device)
```

```
for epoch in range(epochs):
    for i, (data, _) in enumerate(dataloader):
        real = data.to(device)
        b_size = real.size(0)
        label_real = torch.full((b_size,), 1.0, device=device)
        label_fake = torch.full((b_size,), 0.0, device=device)

        # --- Train Discriminator ---
        netD.zero_grad()
        output_real = netD(real)
        lossD_real = criterion(output_real, label_real)
        noise = torch.randn(b_size, nz, 1, 1, device=device)
        fake = netG(noise)
        output_fake = netD(fake.detach())
        lossD_fake = criterion(output_fake, label_fake)
        lossD = lossD_real + lossD_fake
```



```
fake = netG(noise)
```

```
output_fake = netD(fake.detach())
```

```
lossD_fake = criterion(output_fake, label_fake)
```

```
lossD = lossD_real + lossD_fake
```

```
lossD.backward()
```

```
optimizerD.step()
```

```
# --- Train Generator ---
```

```
netG.zero_grad()
```

```
output = netG(fake)
```

```
lossG = criterion(output, label_real)
```

```
lossG.backward()
```

```
optimizerG.step()
```

```
print(f"Epoch [{epoch+1}/{epochs}] Loss_D: {lossD.item():.4f} Loss_G: {lossG.item():.4f}")
```

```
# Show sample results
```

```
with torch.no_grad():
```

```
    fake = netG(fixed_noise).detach().cpu()
```

```
grid = vutils.make_grid(fake, padding=2, normalize=True)
```

```
plt.figure(figsize=(8,8))
```

```
plt.axis("off")
```

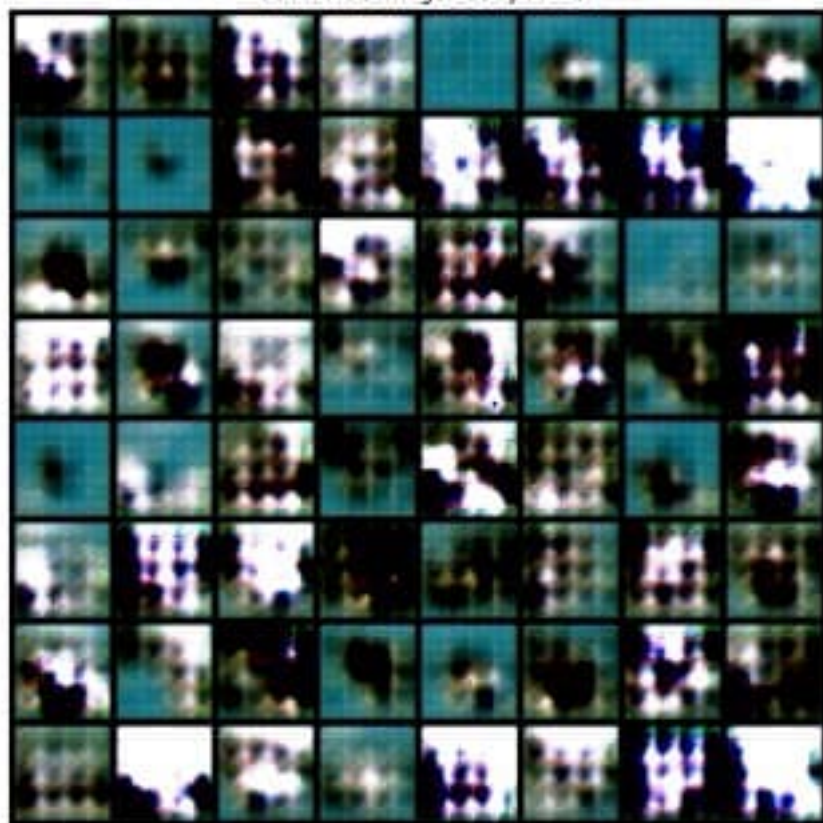
```
plt.title(f"Generated Images at Epoch {epoch+1}")
```

```
plt.imshow(np.transpose(grid, (1,2,0)))
```

```
plt.show()
```

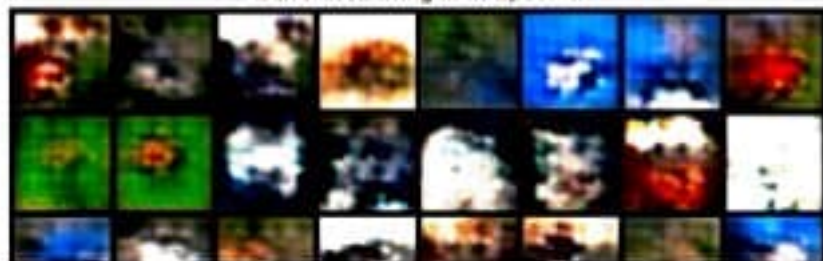
Epoch [1/5] Loss_D: 0.9904 Loss_G: 2.1813

Generated Images at Epoch 1



Epoch [2/5] Loss_D: 0.7218 Loss_G: 2.4231

Generated Images at Epoch 2




```

fake = netG(fixed_noise).detach().cpu()
grid = utils.make_grid(fake, padding=2, normalize=True)
plt.figure(figsize=(8,8))
plt.axis('off')
plt.title('Generated images at Epoch (epoch+1)')
plt.imshow(np.transpose(grid, (1,2,0)))
plt.show()

```

```
print("training complete")
```

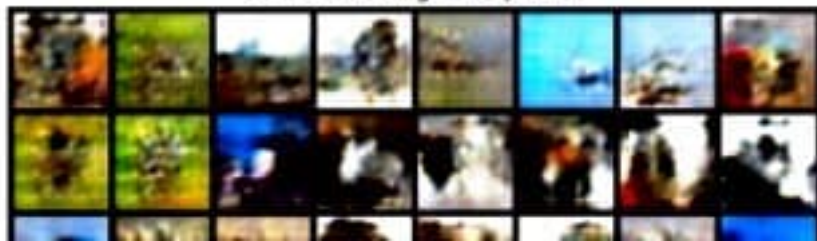
```
Epoch [3/5] Loss_D: 0.4711 Loss_G: 2.0608
```

Generated Images at Epoch 3



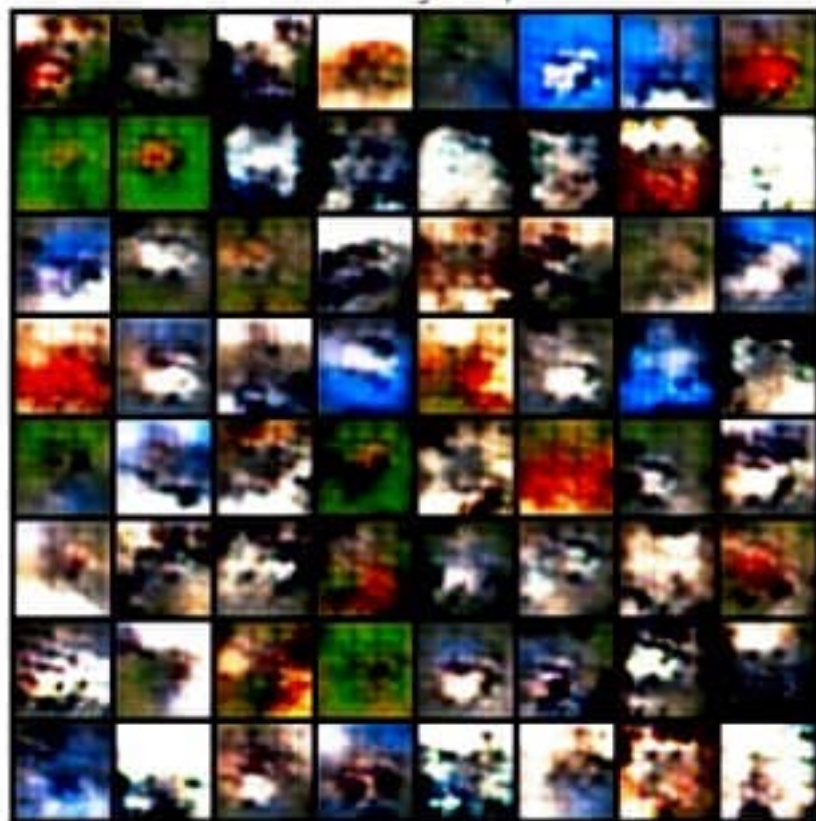
```
Epoch [4/5] Loss_D: 0.7883 Loss_G: 2.0385
```

Generated Images at Epoch 4



Epoch [2/5] Loss_D: 0.7138 Loss_G: 2.4231

Generated Images at Epoch 2



Epoch [3/5] Loss_D: 0.4711 Loss_G: 2.0608

Generated Images at Epoch 3

